18.1 流式传输架构

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~500,000 tokens，输出 ~40,000 tokens（本章合计）

大语言模型生成文本是一个**逐 token** 的过程——每次推理只产生一个 token，然后基于已生成的 token 生成下一个。如果等待整个回复完成才展示给用户，用户可能需要等待数十秒甚至数分钟。**流式传输（Streaming）** 解决了这个问题：模型每生成一个 token（或一小批 token），就立即推送给客户端，让用户能"看到 AI 在打字"。

OpenClaw 的流式传输系统不仅仅是将模型输出转发给用户——它需要处理思考标签的过滤、代码块的安全拆分、消息去重、以及多通道的适配。本节从整体架构出发，介绍 OpenClaw 流式传输系统的核心设计。

18.1.1 Pi Agent Core 事件流 → OpenClaw 事件桥接

OpenClaw 使用的底层 AI Agent 框架（Pi Agent Core / Pi Coding Agent）通过**事件订阅模式**暴露流式数据。 `AgentSession` 提供了 `subscribe` 方法，允许外部代码监听 Agent 执行过程中的各种事件。

OpenClaw 通过 `subscribeEmbeddedPiSession` 函数将这些低层事件桥接到自己的高层抽象：

```typescript
// src/agents/pi-embedded-subscribe.ts（简化）
export function subscribeEmbeddedPiSession(
  params: SubscribeEmbeddedPiSessionParams
) {
  // 1. 初始化状态
  const state: EmbeddedPiSubscribeState = {
    assistantTexts: [],     // 最终收集的助手文本片段
    toolMetas: [],          // 工具调用元数据
    deltaBuffer: "",        // delta 累积缓冲
    blockBuffer: "",        // 块流缓冲
    blockState: { thinking: false, final: false, ... },
    compactionInFlight: false,
    messagingToolSentTexts: [],
    // ...更多状态
  };

  // 2. 创建块分块器（可选）
  const blockChunker = params.blockReplyChunking
    ? new EmbeddedBlockChunker(params.blockReplyChunking)
    : null;

  // 3. 构建上下文对象
  const ctx: EmbeddedPiSubscribeContext = {
    params, state, blockChunker,
    emitBlockChunk, flushBlockReplyBuffer,
    stripBlockTags, emitReasoningStream,
    // ...更多方法
  };

  // 4. 注册事件处理器
  const unsubscribe = params.session.subscribe(
    createEmbeddedPiSessionEventHandler(ctx)
  );

  // 5. 返回控制句柄
  return {
    assistantTexts,     // 可读取的文本片段数组
    toolMetas,          // 工具元数据
    unsubscribe,        // 取消订阅
    isCompacting: () => state.compactionInFlight,
    waitForCompactionRetry: () => { ... },
    didSendViaMessagingTool: () => messagingToolSentTexts.length > 0,
```

```
  };
}
```

核心架构如下：

```
Pi Agent Core                    OpenClaw 事件桥接

┌──────────────┐                 ┌──────────────────────┐
│ AgentSession │   subscribe     │ subscribeEmbedded    │
│              │ ──────────►    │ PiSession            │
│  .subscribe()│                 │                      │
└──────────────┘                 │  ┌─ lifecycle handler │
                                 │  ├─ message handler   │
                                 │  ├─ tool handler      │
                                 │  └─ compaction handler│
                                 └──────────┬───────────┘
                                            │
                                 ┌──────────┼──────────┐
                                 ▼          ▼          ▼
                            onBlockReply  onToolResult  onPartialReply
                            (用户可见)    (verbose模式) (实时预览)
```

订阅参数

`SubscribeEmbeddedPiSessionParams` 定义了订阅者可以配置的所有选项：

```typescript
// src/agents/pi-embedded-subscribe.types.ts
export type SubscribeEmbeddedPiSessionParams = {
  session: AgentSession;         // 底层 Agent 会话
  runId: string;                 // 运行标识
  verboseLevel?: VerboseLevel;   // 输出详细程度
  reasoningMode?: ReasoningLevel;// 推理模式（off/on/stream）
  toolResultFormat?: ToolResultFormat; // 工具结果格式

  // 回调函数
  onBlockReply?: (payload: {     // 块回复回调
    text?: string;
    mediaUrls?: string[];
    audioAsVoice?: boolean;
    replyToId?: string;
  }) => void | Promise<void>;

  onToolResult?: (payload) => void;  // 工具结果回调
  onPartialReply?: (payload) => void;// 部分回复回调（实时更新）
  onReasoningStream?: (payload) => void; // 推理流回调
  onAssistantMessageStart?: () => void;  // 消息开始信号
  onBlockReplyFlush?: () => void;  // 块刷新信号
  onAgentEvent?: (evt) => void;    // 通用事件回调

  // 配置
  blockReplyBreak?: "text_end" | "message_end"; // 块边界
  blockReplyChunking?: BlockReplyChunking; // 分块参数
  enforceFinalTag?: boolean;       // 强制 <final> 标签
};
```

18.1.2 流事件类型：`lifecycle` / `assistant` / `tool`

OpenClaw 将底层事件分为三个**流（Stream）**：

lifecycle 流：生命周期事件

```
// src/agents/pi-embedded-subscribe.handlers.lifecycle.ts
export function handleAgentStart(ctx) {
  emitAgentEvent({
    runId: ctx.params.runId,
    stream: "lifecycle",
    data: { phase: "start", startedAt: Date.now() },
  });
}

export function handleAgentEnd(ctx) {
  emitAgentEvent({
    runId: ctx.params.runId,
    stream: "lifecycle",
    data: { phase: "end", endedAt: Date.now() },
  });
  // Agent 结束时，强制刷新剩余缓冲
  if (ctx.blockChunker?.hasBuffered()) {
    ctx.blockChunker.drain({ force: true, emit: ctx.emitBlockChunk });
    ctx.blockChunker.reset();
  }
}
```

lifecycle 流包含四种事件：

事件	含义	触发时机
`agent_start`	Agent 开始执行	收到用户消息后
`agent_end`	Agent 执行完成	所有工具调用和回复完成
`auto_compaction_start`	自动压缩开始	上下文溢出触发压缩
`auto_compaction_end`	自动压缩结束	压缩完成（可能重试）

assistant 流：助手文本事件

assistant 流是用户最直接关心的——它包含模型生成的文本：

```
// 由 handleMessageUpdate 触发
emitAgentEvent({
  runId: ctx.params.runId,
  stream: "assistant",
  data: {
    text: cleanedText,   // 完整的累积文本
    delta: deltaText,    // 本次新增的文本片段
    mediaUrls: hasMedia ? mediaUrls : undefined,
  },
});
```

> **衍生解释——Delta vs Full Text**
>
> 流式传输有两种数据模式：**Delta 模式**只发送每次的增量部分（"Hello" → " world" → "!"），客户端需要自己拼接；**Full 模式**每次发送累积的完整文本（"Hello" → "Hello world" → "Hello world!"）。OpenClaw 同时提供了 `delta` 和 `text` 字段，让不同的消费者可以按需选择。

tool 流：工具执行事件

```
// 工具开始
emitAgentEvent({
  stream: "tool",
  data: { phase: "start", name: toolName, toolCallId, args },
});

// 工具进行中（部分结果）
emitAgentEvent({
  stream: "tool",
  data: { phase: "update", name: toolName, toolCallId, partialResult },
});

// 工具完成
emitAgentEvent({
  stream: "tool",
  data: { phase: "result", name: toolName, toolCallId, isError, result },
});
```

compaction 流：压缩事件

```
// 压缩开始
emitAgentEvent({ stream: "compaction", data: { phase: "start" } });

// 压缩结束
emitAgentEvent({
  stream: "compaction",
  data: { phase: "end", willRetry: Boolean(evt.willRetry) },
});
```

18.1.3 原始流处理（src/agents/pi-embedded-subscribe.raw-stream.ts）

OpenClaw 提供了一个**调试级**的原始流记录器，可以将所有底层事件原样写入 JSONL 文件：

```
// src/agents/pi-embedded-subscribe.raw-stream.ts
const RAW_STREAM_ENABLED = isTruthyEnvValue(
  process.env.OPENCLAW_RAW_STREAM
);
const RAW_STREAM_PATH = process.env.OPENCLAW_RAW_STREAM_PATH?.trim()
  || path.join(resolveStateDir(), "logs", "raw-stream.jsonl");

export function appendRawStream(payload: Record<string, unknown>) {
  if (!RAW_STREAM_ENABLED) return;

  if (!rawStreamReady) {
    rawStreamReady = true;
    try {
      fs.mkdirSync(path.dirname(RAW_STREAM_PATH), { recursive: true });
    } catch { /* 忽略 */ }
  }

  try {
    void fs.promises.appendFile(
      RAW_STREAM_PATH,
      `${JSON.stringify(payload)}\n`
    );
  } catch { /* 忽略 */ }
}
```

通过设置环境变量 `OPENCLAW_RAW_STREAM=1` 启用后，每个文本事件都会被记录：

```
{"ts":1708243791000,"event":"assistant_text_stream","runId":"run_abc","evtType":"text_d
{"ts":1708243791050,"event":"assistant_text_stream","runId":"run_abc","evtType":"text_d
{"ts":1708243791100,"event":"assistant_message_end","runId":"run_abc","rawText":"Hello
```

> **衍生解释——JSONL（JSON Lines）格式**
>
> JSONL 是一种行分隔的 JSON 格式——每行一个完整的 JSON 对象。与标准 JSON 数组不同，JSONL 文件可以增量追加而无需重写整个文件，非常适合日志记录场景。大多数数据分析工具（如 `jq`）都原生支持 JSONL。

事件处理器路由

底层事件通过 `createEmbeddedPiSessionEventHandler` 路由到对应的处理器：

```ts
// src/agents/pi-embedded-subscribe.handlers.ts
export function createEmbeddedPiSessionEventHandler(ctx) {
  return (evt: EmbeddedPiSubscribeEvent) => {
    switch (evt.type) {
      case "message_start":    handleMessageStart(ctx, evt);    return;
      case "message_update":   handleMessageUpdate(ctx, evt);   return;
      case "message_end":      handleMessageEnd(ctx, evt);      return;
      case "tool_execution_start":
        handleToolExecutionStart(ctx, evt).catch(/* 忽略 */);
        return;
      case "tool_execution_update": handleToolExecutionUpdate(ctx, evt); return;
      case "tool_execution_end":    handleToolExecutionEnd(ctx, evt);    return;
      case "agent_start":           handleAgentStart(ctx);               return;
      case "agent_end":             handleAgentEnd(ctx);                 return;
      case "auto_compaction_start": handleAutoCompactionStart(ctx);      return;
      case "auto_compaction_end":   handleAutoCompactionEnd(ctx, evt);   return;
    }
  };
}
```

`tool_execution_start` 使用了 `async` 处理器并 `.catch()` 忽略错误——这是因为工具开始时可能触发输入提示（typing indicator），这是"尽力而为"的操作，不应阻塞后续流程。

本节小结

1. **流式传输**让用户在模型生成过程中就能看到文本，极大提升了交互体验。
2. OpenClaw 通过 `subscribeEmbeddedPiSession` 将底层 Pi Agent Core 的事件流**桥接**为高层的三类流：lifecycle（生命周期）、assistant（助手文本）、tool（工具执行）。
3. **事件路由**使用 switch-case 模式分发到四类处理器：生命周期、消息、工具、压缩。
4. **原始流记录器**提供了调试级的事件追踪能力，通过环境变量按需启用。
5. 每个事件流都同时通过 `emitAgentEvent` （全局事件总线）和回调函数两种方式对外暴露。